SE EXPERIMENT 1

Aim: Write a detailed Problem Statement for any case study. Justify which process model would be best suited to apply on it.

Title: Smart Parking Management System (SPMS)

Background: As urban populations grow, the number of vehicles increases exponentially, causing extreme congestion in commercial, residential, and corporate multi-storey parking facilities. Traditional parking management relies heavily on manual ticketing, security guard oversight, and a "first-come, first-served" cruising model.

The Problem:

1. **Inefficient Space Utilization & Cruising Time:** Drivers waste an average of 10 to 20 minutes circling multiple floors looking for vacant spots. This leads to high fuel consumption, unnecessary carbon emissions, and driver frustration.
2. **Manual Bottlenecks:** Parking staff struggle to keep track of real-time spot occupancy across multiple levels, resulting in underutilized floors while the ground floor experiences heavy bottlenecks.
3. **Lack of Data Visibility:** Facility managers lack real-time analytics regarding peak hours, average occupancy durations, or revenue leakages.
4. **Hardware-Software Disconnect:** Many existing systems treat sensor arrays (hardware) and booking software as isolated silos rather than an integrated ecosystem.

The Solution: The SPMS aims to bridge this gap by deploying an automated, online software platform integrated with IoT-based proximity/ultrasonic sensors at every single parking unit. The software allows parking staff to immediately allocate an optimal, system-verified vacant spot to incoming customers upon entry, completely eliminating the need for drivers to look around blindly.

Core Modules & System Architecture

To make the theoretical design robust, the system architecture is divided into three concrete layers:

- **The Driver Portal (Frontend Concept):** A user-facing application interface where drivers can register their vehicles, check real-time parking lot occupancy maps before arrival, view allocated slots, and view historical billing data.
- **The Operator/Admin Dashboard:** A high-privilege console used by parking lot management to monitor live parking configurations, override system locks, analyze peak occupancy hours, and audit generated revenue.

- **The Core Logic Engine & Database (Backend Layout):** The central brain that maintains the system state. It handles incoming requests, updates database records when a slot transitions from Available to Occupied, and calculates precise financial tariffs based on check-in and check-out timestamps.

System Flow & Business Logic

1. **Check-In Phase:** A vehicle arrives at the gate. The system captures the vehicle number, checks the database for available slots, updates the state of the selected slot from Available to Occupied, and prints/generates a digital ticket containing a unique ID and an Entry Timestamp.
2. **In-Lot Phase:** The system continuously monitors the database. The slot remains locked under the active vehicle's profile.
3. **Check-Out Phase:** Upon exiting, the ticket ID is scanned. The system fetches the Entry Timestamp, logs the current Exit Timestamp, and passes both values into a billing function. Once the transaction is cleared, the system resets the slot state back to Available.

The Selected Process Model: The Incremental Model

The **Incremental Process Model** breaks the software product down into multiple, linear development cycles called **increments**. Each increment delivers a fully functioning portion of the software, building sequentially upon the previous release until the entire system is complete.

For this project, the theoretical development is mapped into **three distinct increments**:

Increment 1: Core Slot Allocation Engine (The Foundation)

- **Focus:** Core database schemas and structural data flows.
- **Deliverables:** Designing the Users, Slots, and Tickets tables. Creating the algorithmic backend function allocateSlot() and the state machine logic that transitions a parking slot between states (Available, Occupied, Under Maintenance).
- **Testing Focus:** Basic structural verification to ensure database transactions don't fail under concurrent arrival requests.

Increment 2: User & Administrative Interfaces (The Interaction Layer)

- **Focus:** User authentication, dashboards, and access control.
- **Deliverables:** Building the driver interface (UI wireframes for viewing empty spots) and the admin console (UI for monitoring total lot statistics).
- **Integration:** Attaches these interfaces directly onto the core engine built in Increment 1, allowing users to physically trigger the database state changes.

Increment 3: Automated Billing & Analytical Reporting (The Premium Layer)

- **Focus:** Financial transactions, duration math, and reports.
- **Deliverables:** Writing the calculateTariff(entryTime, exitTime) function, handling digital payment success/failure state paths, and generating text-based receipts.
- **Final Assembly:** Completes the product, leaving you with a fully realized, multi-layered software architecture.

Why the Incremental Model is the Most Suitable

Choosing the Incremental Model over other frameworks like Waterfall, Prototype, or Agile Scrum provides critical advantages for a theoretical parking system design:

- **Clean Structural Division for Academic Reports:** A parking system naturally splits into isolated, modular operational layers (Allocation ➔ Interfaces ➔ Billing). The Incremental Model mirrors this perfectly, allowing you to write your SRS and design your Level 1 and Level 2 Data Flow Diagrams (DFDs) page-by-page, module-by-module, keeping your lab documentation extremely clean.
- **Simplified Estimation and Testing Metrics:** In later practicals, trying to evaluate a massive, fully integrated system all at once is overwhelming. With the Incremental Model, you can easily isolate a single function from Increment 1 (like slot state toggling) or Increment 3 (the tariff equation) to draw control flow graphs and write exact test cases without confusing your structural paths.
- **Ideal for Team Task Delegation:** If you are collaborating with peers, this model eliminates conflicting design overlaps. One person can own the theoretical architectural design for the foundational database layout (Increment 1), while another designs the visual wireframes and actor use cases (Increment 2) concurrently.
- **Mitigates Design Over-Complexity:** Unlike evolutionary or prototype models which can loop endlessly, the linear progress of each increment ensures that you stay on schedule to hit every deadline listed in your college lab manual.